

Designed for fresher & junior DevOps interviews | Reference guide, not a memorisation sheet

Table of Contents

Section 1 — Terraform Apply & Plan Failures	Q1–Q4
Section 2 — State & Locking Issues	Q5–Q8, Q16–Q18, Q24
Section 3 — CI/CD & Variables	Q9–Q10
Section 4 — Networking & Application Access	Q11–Q15
Section 5 — Environment & Destroy Issues	Q19–Q20
Section 6 — Troubleshooting, Drift & Security	Q21–Q25

Section 1 — Terraform Apply & Plan Failures

Q1. Terraform apply suddenly fails. What will you check first?

- Read the full error output — Terraform error messages usually name the exact resource and provider call that failed.
- Check AWS credentials/IAM permissions — expired tokens, insufficient policies, or wrong profile are the #1 cause.
- Verify provider version constraints in versions.tf — a provider upgrade can introduce breaking changes.
- Look for state lock — a prior crashed run may have left a DynamoDB lock entry; use `terraform force-unlock`` after confirming safety.
- Check network connectivity — if running in CI, the runner may not reach the AWS endpoint (VPC, proxy, or firewall issue).
- Review recent code changes with `git diff`` — a typo or removed variable can silently break an apply.

■ *Interview tip: Always mention reading the error first — jumping to fixes without reading output is a red flag for interviewers.*

Q2. Terraform plan shows unexpected infrastructure changes. Why?

- State drift — someone manually changed a resource in AWS Console; Terraform detects the delta and plans to revert it.
- Provider upgrade — newer provider versions may change default attribute values, triggering diffs.
- Variable or locals change — an environment-specific tfvars was not applied, causing different resolved values.
- Data source refresh — `data` blocks always re-fetch at plan time; a changed AMI ID or subnet CIDR causes cascading diffs.
- Resource dependency change — adding/removing a `depends\_on` can alter the order and trigger re-creation.
- Sensitive attribute masking — Terraform may show `(known after apply)` incorrectly flagging a no-op as a change.

■ Key command: `terraform plan -refresh=false` isolates code changes from state drift changes.

Q3. Infrastructure is created partially. What could be wrong?

- Mid-run error hit a resource that depends on an earlier failed resource — Terraform stops but keeps what succeeded in state.
- Timeout on a slow resource (e.g., RDS) — default provider timeouts may not be sufficient; set custom `timeouts {}` blocks.
- IAM permission missing for a specific resource type — creates some resources, fails on restricted ones.
- Race condition — a resource was created before its dependency was ready (missing `depends\_on` or implicit reference).
- State file was unlocked prematurely — a second run started and created partial resources on top.

■ Recovery: Run `terraform apply` again after fixing the root cause — Terraform is idempotent and will only create missing resources.

Q4. Terraform execution hangs indefinitely. What will you investigate?

- Resource is stuck in a transitional AWS state (e.g., EC2 in `pending`, RDS in `modifying`) — check AWS Console.
- State lock is held by a previous run — check DynamoDB table for a lock entry; use `terraform force-unlock` if confirmed stale.
- A `local-exec` or `remote-exec` provisioner is waiting for SSH/WinRM that never connects — check security group rules.
- Network timeout to AWS API — VPC endpoint or proxy misconfiguration can cause silent hangs.
- Resource creation is waiting for a dependency that never completes (circular or implicit dependency).

■ Set `TF_LOG=DEBUG` to see exactly which API call is blocking.

Section 2 — State & Locking Issues

Q5. State Lock error appears during deployment. Why?

- Another `terraform apply` or `plan` is actively running and holds the DynamoDB lock.
- A previous run crashed without releasing the lock — the lock entry remains in DynamoDB.
- Two CI jobs triggered simultaneously for the same workspace (no pipeline concurrency control).
- Lock ID mismatch — manually edited state or switched backends without proper migration.

■ *Fix: Confirm no active run exists, then `terraform force-unlock`. Never force-unlock if a run is actually in progress.*

Q6. Terraform cannot access Remote State. What could be the reason?

- IAM role/user lacks `s3:GetObject` permission on the state bucket.
- S3 bucket name or key path in `backend {}` block is incorrect.
- Bucket is in a different AWS region than what the provider/env is configured for.
- VPC endpoint for S3 not configured — runner inside a private subnet cannot reach S3.
- Bucket versioning or ACL change blocked access (e.g., Block Public Access accidentally applied to internal role).
- KMS key used for bucket encryption was rotated or deleted — decryption fails silently.

Q7. S3 Backend is configured but State File is missing. What happened?

- First `terraform init` was never followed by `terraform apply` — state file is created on first apply, not on init.
- Wrong key path in backend config — the file exists but at a different S3 prefix.
- State file was accidentally deleted (no S3 versioning enabled to recover it).
- Workspace mismatch — state exists under `env:/prod/terraform.tfstate` but you are in default workspace.
- Backend was migrated but `-migrate-state` flag was not passed — old state was not copied.

■ *Best practice: Always enable S3 versioning on state buckets so accidental deletions are recoverable.*

Q8. DynamoDB locking is not working. Why?

- DynamoDB table does not have a primary key named exactly `LockID` (case-sensitive).
- IAM policy missing `dynamodb:PutItem`, `dynamodb:GetItem`, or `dynamodb:DeleteItem` on the lock table.
- Table is in a different region than the S3 bucket — backend config `region` must match both.
- `dynamodb_table` parameter in backend config is misspelled or omitted entirely.
- Table was created with on-demand capacity but region capacity is throttled — lock writes silently fail.

Q16. Terraform State File gets corrupted. What will you do?

- Do not panic or delete — stop all Terraform operations immediately.
- Restore from S3 versioned backup: `aws s3api get-object --version-id --bucket --key terraform.tfstate`.
- If no versioning, use `terraform state list` to check what is partially readable.
- For severe corruption, rebuild state with `terraform import` for each resource — slow but reliable.
- After recovery, run `terraform plan` to verify state matches actual infrastructure before any apply.
- Post-incident: enable S3 versioning, cross-region replication, and restrict direct state file access via IAM.

■ *Prevention is everything here — a corrupted state with no backup is one of the worst Terraform incidents.*

Q17. Someone manually changes AWS resources after deployment. What happens?

- Terraform state no longer matches real infrastructure — this is called state drift.
- Next `terraform plan` will show changes to revert the manual modification back to the declared state.
- If you run `terraform apply`, Terraform will overwrite the manual change.
- Use `terraform refresh` (or `terraform apply -refresh-only`) to update state to match current reality without applying code changes.
- Prevent recurrence: enforce SCPs/IAM policies to block console changes in production; use CloudTrail to audit who changed what.

■ *Interview point: Distinguish between `terraform refresh` (updates state from real infra) vs `terraform apply` (makes real infra match code).*

Q18. Multiple engineers run Terraform simultaneously. What issues can occur?

- Race condition on state file — both reads happen before either write, causing one run to overwrite the other's changes.
- State lock contention — if DynamoDB locking is configured correctly, second run will block; if not, both proceed and corrupt state.
- Duplicate resource creation — two engineers create the same resource in different apply runs before state syncs.
- Partial applies — one run succeeds halfway, the other starts with a stale state view.

■ *Solution: Use remote backends with state locking + Terraform Cloud/Atlantis for serialized, reviewed applies.*

Q24. Why is State Locking critical in production?

- Prevents concurrent applies from corrupting the state file — state corruption without a backup is unrecoverable.
- Ensures only one operation modifies infrastructure at a time — avoids duplicate resource creation or race conditions.
- Provides audit trail — DynamoDB lock entries record who locked, when, and from where.
- Without locking, a CI pipeline and a developer running apply simultaneously will produce unpredictable results.
- In regulated environments, state locking is part of change management compliance.

Section 3 — CI/CD & Variables

Q9. Terraform code works locally but fails in Jenkins. Why?

- AWS credentials not injected into Jenkins — local uses `~/.aws/credentials`; Jenkins needs env vars or IAM instance profile.
- Different Terraform version on Jenkins vs local — use `.terraform-version` file with `tfenv` or pin version in pipeline.
- Missing environment-specific tfvars — local uses `-var-file=dev.tfvars`; Jenkins job may not pass the correct var file.
- Network access differences — Jenkins agent may be in a private subnet without internet/VPC endpoint access.
- `terraform init` not run in Jenkins before `plan`/`apply` — providers and modules need to be downloaded.
- Working directory mismatch — Jenkins checks out code to a different path than expected by relative module references.

Q10. Variables are not getting applied correctly. What will you verify?

- Variable precedence order: CLI `-var` > `.auto.tfvars` > `terraform.tfvars` > environment vars (`TF_VAR_name`) > default values.
- Verify the correct `-var-file` is being passed in the command.
- Check for typos in variable names — Terraform will use the default value silently if a variable name doesn't match.
- Confirm `sensitive = true` variables are not being masked in output but are actually resolved correctly.
- For locals, check that `local.variable` is not referencing a variable that evaluates to `null`.
- Run `terraform console` and type the variable name to see its resolved value interactively.

Section 4 — Networking & Application Access

Q11. EC2 instances are created but application is inaccessible. Why?

- Security Group inbound rules not allowing traffic on the application port (e.g., 8080, 443).
- Instance is in a private subnet with no NAT Gateway or bastion — application not reachable from internet.
- User data script failed — application never started; check `/var/log/cloud-init-output.log`.
- IAM instance profile missing — application cannot access S3/Secrets Manager at startup.
- AMI used does not have the application pre-installed — check Terraform `ami` variable value.
- Health check misconfigured on Load Balancer — traffic not forwarded even though instance is healthy at OS level.

Q12. Security Groups are created but traffic is blocked. What happened?

- Inbound rule created but on wrong port — e.g., 80 allowed but app runs on 8080.
- CIDR block is incorrect — `10.0.0.0/8` instead of `0.0.0.0/0` for internet-facing, or wrong VPC CIDR for internal.
- Security Group attached to wrong resource — Terraform `vpc\_security\_group\_ids` references a different SG.
- NACLs (Network ACLs) are blocking traffic at subnet level — SGs are stateful, NACLs are stateless and need explicit return rules.
- Egress rules missing — if you replaced default egress, outbound responses are blocked.

■ *Remember: Security Groups are stateful (return traffic auto-allowed). NACLs are stateless (return traffic needs explicit allow rule).*

Q13. Load Balancer is healthy but users cannot access the application. Why?

- DNS not updated — Route53 record still points to old IP or ALB DNS name is not propagated yet.
- Target group port mismatch — ALB listens on 443 but forwards to port 80; application expects HTTPS.
- SSL certificate not attached or expired on HTTPS listener.
- Security Group on ALB does not allow inbound 443/80 from `0.0.0.0/0`.
- Listener rules misconfigured — default action is `Fixed response 503` instead of `Forward to target group`.
- Targets are healthy but application returns 4xx/5xx — ALB health check passes but app-level errors block users.

Q14. Route Tables are created but connectivity fails. What will you check?

- Route table not associated with the correct subnet — association is a separate resource (`aws\_route\_table\_association`).
- Route to Internet Gateway missing for public subnets — `0.0.0.0/0 -> igw-xxxx` must be explicit.
- Private subnet route to NAT Gateway missing — `0.0.0.0/0 -> nat-xxxx` required for outbound internet.
- Subnet is using the default VPC route table, not the custom one created by Terraform.
- VPC peering route missing — if connecting two VPCs, each side needs a route to the other's CIDR.

Q15. Auto Scaling resources are not getting created. Why?

- Launch Template or Launch Configuration referenced by ASG does not exist — creation order dependency issue.
- IAM Service-Linked Role for Auto Scaling not present in the account — required on first ASG creation.
- Min/Max/Desired capacity values are invalid (e.g., desired > max).
- VPC subnet IDs in ASG config are wrong or subnets are at capacity.
- Scaling policy references a CloudWatch metric that doesn't exist yet — policy creation fails silently.

Section 5 — Environment & Destroy Issues

Q19. Terraform destroy fails midway. What next?

- Read the error — usually a resource has a dependency that must be destroyed first (e.g., S3 bucket not empty).
- For S3: empty the bucket first with `aws s3 rm s3://bucket --recursive`, then re-run destroy.
- For EKS/RDS: ensure all dependent resources (node groups, read replicas) are removed before the parent.
- Re-run `terraform destroy` — it is idempotent; already-destroyed resources are skipped, only remaining ones are attempted.
- If a resource is stuck, use `terraform state rm` to remove it from state, then manually delete from AWS Console.

■ *Never just remove resources from state to 'fix' a destroy failure without manually verifying the resource is actually gone.*

Q20. Infrastructure works in Dev but fails in Prod. Why?

- Different variable values — `prod.tfvars` has different instance types, CIDR ranges, or feature flags.
- IAM permission difference — prod AWS account has stricter SCPs or resource-based policies.
- Missing resource limits in prod — AWS service quotas (e.g., max VPCs, EC2 limits) may be hit in prod but not dev.
- Prod uses different Terraform workspace or module version — not in sync with dev.
- Data sources return different values — an AMI or subnet ID that exists in dev doesn't exist in prod region.
- Prod account has mandatory tags enforced by SCP — missing tags cause resource creation to fail.

Section 6 — Troubleshooting, Drift & Security

Q21. How would you troubleshoot a failed Terraform deployment?

- Step 1 — Read the error output completely; identify the failing resource and the exact error message.
- Step 2 — Enable debug logging: `export TF_LOG=DEBUG` for verbose API call traces.
- Step 3 — Check state file consistency: `terraform show` to see what Terraform thinks exists.
- Step 4 — Validate AWS Console — does the resource exist in a weird state (e.g., `deleting`).
- Step 5 — Isolate the failing resource using `-target=resource.name` to apply/plan just that resource.
- Step 6 — Check IAM permissions for the exact API call that failed (look for `AccessDenied` in logs).
- Step 7 — Review git history for recent changes that could have introduced the issue.

Q22. How would you identify Terraform state drift?

- Run `terraform plan -refresh=true` (default) — any diff output indicates drift between state and real infra.`
- Use `terraform apply -refresh-only` to pull real infrastructure state into the state file without changing anything.`
- Use AWS Config Rules or CloudTrail to detect changes made outside Terraform.
- Set up scheduled Terraform plan runs in CI (drift detection pipeline) — alert on non-empty plan output.
- Tools like `driftctl` or Terraform Cloud provide automated drift detection and reporting.`

■ *Drift detection pipelines are a production best practice — don't wait for a failed apply to discover drift.*

Q23. How would you secure Terraform State Files?

- Store state in S3 with server-side encryption (SSE-KMS) — never store locally in production.
- Enable S3 bucket versioning — recover from accidental deletions or corruption.
- Restrict S3 bucket access with IAM policies — only Terraform runners and ops team should have access.
- Enable S3 access logging and CloudTrail for audit of all state file reads/writes.
- Block public access on the S3 bucket — state files contain sensitive resource IDs and outputs.
- Use Terraform Cloud/Enterprise for built-in encrypted state management with RBAC.
- Never commit state files to Git — add `*.tfstate` and *.tfstate.backup` to .gitignore`.`

Q25. What are your first 5 checks during a Terraform production outage?

- Check 1 — Verify if Terraform was the cause: review recent apply logs and git commits for changes deployed in the last window.
- Check 2 — Look for state lock: a stuck apply may be mid-execution; do not run anything until confirmed.
- Check 3 — Check AWS Service Health Dashboard and CloudTrail for AWS-side issues or API errors.
- Check 4 — Run `terraform plan` (read-only, no apply) to see the current diff between state and real infra.`
- Check 5 — Check CloudWatch alarms, Datadog/Grafana dashboards, and application logs to isolate whether the issue is infra or app-level.

■ *In a real outage, communicate status first, investigate second — always update your incident channel before diving deep.*

Prepared for DevOps interview readiness (0–3 Years) • Practice by reasoning through answers, not memorising them •
Pair this with hands-on Terraform runs on your ApnaKart project for maximum retention.